

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524281
Name	KAVIYA.G

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I **KAVIYA.G(192524281)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:KAVIYA.G

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

SOLUTION:

Assignment: AI Search Algorithms for Emergency Medicine Delivery Using Drones

Scenario Overview

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic, meaning that roads and flying routes may suddenly become blocked due to floods, fallen trees, or damaged infrastructure. In addition, weather conditions such as heavy rain, strong winds, and storms affect the drone's travel speed and energy consumption, causing variable movement costs.

The drone has access to:

- A heuristic estimate (straight-line distance to the destination)
- Partial real-time updates about blocked paths
- Variable travel costs depending on weather and terrain

The primary objective is to deliver medicines quickly, safely, and with minimum delay, while adapting to changing environmental conditions.

(i) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Working of Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next node that appears to be closest to the destination according to the heuristic function.

It uses only:

[
 $f(n)=h(n)$
]

Where:

- $h(n)$ = Estimated straight-line distance from the current node to the destination.

Unlike other algorithms, Greedy Search ignores the cost already spent travelling.

Behaviour in the Flood Scenario

The drone always flies toward the location that appears nearest to the hospital or relief center.

For example,

- Destination is 12 km away.
- Route A appears only 4 km from the destination.
- Route B appears 6 km away.

Greedy Search immediately selects Route A because it has the smallest heuristic value.

However, after travelling, the drone may discover:

- Flooded areas
- Strong winds
- Blocked bridges
- No-fly zones

Since these conditions were not considered initially, the drone must search again, causing delays.

Strengths of Greedy Best-First Search

1. Very Fast Decision Making

Greedy Search expands only the most promising node.

Therefore, route selection is extremely fast.

This is useful during emergency medicine delivery where every second matters.

2. Lower Memory Usage

It explores fewer nodes than many other algorithms.

Hence, computational requirements are small, making it suitable for drones with limited onboard processors.

3. Simple Implementation

The algorithm only needs the heuristic estimate.

Its implementation is easy and requires less computation.

Weaknesses under Uncertainty

1. Ignores Actual Travel Cost

Greedy Search does not consider:

- Wind resistance
- Battery usage
- Terrain difficulty
- Flood severity

Therefore, it may choose a path that looks short but is actually expensive.

2. Poor Performance with Dynamic Obstacles

Suppose a bridge suddenly collapses.

The algorithm may have already committed to that direction.

It must restart the search, increasing delivery time.

3. Does Not Guarantee the Best Route

Greedy Search is **not optimal**.

It often produces the first promising path instead of the safest or least expensive one.

4. High Risk During Emergencies

Because safety factors are ignored, the drone may repeatedly enter dangerous regions before rerouting.

Overall Evaluation of Greedy Search

Greedy Best-First Search provides **very fast navigation**, but under uncertain flood conditions it may repeatedly choose unsafe or blocked routes, reducing reliability.

(ii) Explain how A* Search would handle the same situation, including how it balances cost and heuristic.

Working of A* Search

A* evaluates every node using both

- Actual travel cost
- Estimated remaining distance

It uses the evaluation function

$$f(n) = g(n) + h(n)$$

Where

- **$g(n)$** = Actual cost travelled so far
- **$h(n)$** = Estimated distance remaining
- **$f(n)$** = Total estimated cost

Behaviour in the Flood Scenario

Suppose two routes exist.

Route A

Current travel cost

$$g(n) = 14$$

Remaining distance

$$h(n) = 5$$

Total

$$f(n) = 19$$

Route B

Current travel cost

$$g(n) = 10$$

Remaining distance

$$h(n) = 10$$

Total

$$f(n) = 20$$

Although Route B has travelled less, Route A still has the smaller total estimated cost.

Therefore, A* selects Route A.

Unlike Greedy Search, A* also considers

- Fuel consumption
 - Wind resistance
 - Flood depth
 - Battery usage
 - Risky regions
-

Balancing Cost and Heuristic

The major strength of A* is balancing

Actual Cost

How much battery, fuel and time have already been spent.

Estimated Future Cost

How far the destination still is.

Because both values are considered, the chosen path is usually both

- Fast
- Safe
- Cost-effective

Advantages of A*

Finds Optimal Route

If the heuristic is admissible, A* always finds the minimum-cost path.

Better Safety

Dangerous routes receive higher travel costs.

Therefore, safer paths naturally become preferred.

Adapts to Variable Costs

When weather changes,

- Wind speed increases
- Heavy rain occurs
- Flying becomes difficult

the movement cost $g(n)$ changes.

A* automatically updates its decision.

Better Resource Utilization

Battery power is conserved because unnecessary detours are avoided.

Limitations

- Requires more memory.
- More computation than Greedy Search.
- Slightly slower during route planning.

However, the improved route quality usually compensates for the additional computation.

(iii) Analyze the impact of heuristic accuracy on both algorithms.

The heuristic function estimates the remaining distance to the goal.

Its accuracy greatly influences both search algorithms.

Accurate Heuristic

When the heuristic closely matches the real distance,

Greedy Search

- Quickly reaches the destination.
- Explores fewer nodes.
- Makes fewer wrong decisions.

A*

- Finds the optimal route.
- Reduces unnecessary exploration.
- Improves efficiency.

Poor Heuristic

When the heuristic is inaccurate,

Greedy Search

Performance degrades significantly.

It may repeatedly choose

- Flooded roads
- Unsafe regions
- Dead ends

because it trusts the incorrect estimate.

A*

Although search becomes slower,

A* still considers the actual cost $g(n)$.

Therefore, it can often recover from heuristic errors and still produce an optimal solution.

Comparison

Heuristic Quality	Greedy Search	A* Search
Accurate	Very fast	Fast and optimal
Moderately accurate	Acceptable	Very reliable
Poor	Performs poorly	Slower but still effective

Conclusion

A* is less dependent on heuristic quality because it combines both actual and estimated costs, while Greedy Search relies entirely on the heuristic.

(iv) Discuss how dynamic changes (blocked paths) affect search performance.

In flood situations, the environment continuously changes.

Examples include

- Bridges collapsing
- Roads submerged
- Trees blocking routes
- Sudden storms
- New no-fly zones

Effect on Greedy Best-First Search

When a chosen route becomes blocked,

- Search restarts.
- Previously explored nodes become useless.
- Multiple rerouting operations occur.

This increases

- Delivery time
- Battery usage
- Computational overhead

Effect on A*

When obstacles appear,

A* recalculates both

- Current travel cost
- Remaining estimated cost

Blocked paths receive extremely high costs or become inaccessible.

The algorithm naturally chooses an alternative path.

Although replanning requires additional computation, the resulting path remains safer and closer to optimal.

Comparison

Situation	Greedy Search	A* Search
Sudden obstacle	Often restarts search	Replans efficiently
Variable weather	Poor handling	Handles naturally
Route reliability	Lower	Higher
Safety	Moderate	High

(v) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Recommended Algorithm: A* Search

For emergency medicine delivery in flood-affected areas, *A Search** is the most suitable algorithm.

Justification

1. Better Decision Making

A* evaluates

- Current travel cost
- Future estimated distance

Therefore, it makes balanced routing decisions.

2. Higher Safety

Dangerous flooded regions can be assigned higher movement costs.

The algorithm naturally avoids unsafe routes.

3. Optimal Path

Unlike Greedy Search,

A* guarantees the minimum-cost route when an admissible heuristic is used.

This improves delivery efficiency.

4. Handles Dynamic Environments

When weather changes or roads become blocked,

A* updates the path using new cost information.

This makes it highly suitable for disaster response.

5. Better Resource Management

Efficient routing reduces

- Battery consumption
- Flight time
- Risk of mission failure

6. Reliable Emergency Response

Although A* requires slightly more computation, modern AI drones can process these calculations quickly.

The small increase in computation is justified by

- Higher reliability
- Better safety
- Reduced delivery failures
- Faster successful medicine delivery

Final Comparison

Feature	Greedy Best-First Search	A* Search
Evaluation Function	$h(n)$	$g(n)+h(n)$
Uses Actual Cost	No	Yes
Uses Heuristic	Yes	Yes

Optimal Solution	No	Yes (with admissible heuristic)
Speed	Very Fast	Fast
Memory Usage	Low	Higher
Dynamic Environment	Poor	Good
Safety	Moderate	High

Conclusion

In a flood-affected emergency environment, the AI drone must make rapid yet safe routing decisions while adapting to blocked paths and changing weather. Greedy Best-First Search provides quick decisions but ignores travel cost and may repeatedly select unsafe or blocked routes. In contrast, A* combines the actual travel cost with the heuristic estimate, enabling it to choose efficient, low-risk, and near-optimal paths even when conditions change. Although A* requires more computational resources, its superior adaptability, optimality, and safety make it the best choice for real-time emergency medicine delivery, where reliability and successful mission completion are more important than simply making the fastest initial decision.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Recommend the best approach and justify your choice based on scalability and adaptability

SOLUTION:

Optimization of Smart City Traffic Signals Using Artificial Intelligence

Scenario Overview

A smart city has implemented an **Artificial Intelligence (AI)-based traffic management system** to optimize traffic signal timings across hundreds of intersections. The system continuously collects real-time traffic information from cameras, sensors, GPS devices, and connected vehicles. Its primary objective is to reduce:

- Total waiting time at intersections
- Fuel consumption
- Traffic congestion

However, the system faces several challenges:

- The search space is extremely large because there are hundreds of traffic signals, each with multiple timing combinations.
- Traffic conditions change dynamically due to accidents, road closures, weather, and peak-hour traffic.
- The optimization algorithm may become trapped in locally optimal solutions instead of finding the global optimum.

Therefore, intelligent optimization techniques are required to efficiently determine the best signal timings while adapting to changing traffic conditions.

(i) Formulate this as an Optimization Problem and Explain Why Traditional Search is Inefficient

Formulation as an Optimization Problem

The traffic signal optimization problem can be represented as an optimization problem where the AI system searches for the best combination of signal timings.

Decision Variables

The AI adjusts:

- Green light duration
- Red light duration
- Yellow light duration
- Signal sequence
- Coordination between neighboring intersections

Objective Function

The objective is to minimize

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion
- Vehicle emissions
- Overall travel time

Subject to the following constraints:

- Minimum and maximum green signal duration
- Pedestrian crossing requirements
- Emergency vehicle priority
- Traffic safety regulations
- Maximum queue length

The optimal solution is the signal timing plan that minimizes the total traffic cost while satisfying all constraints.

Why Traditional Search is Inefficient

Traditional search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), or Exhaustive Search evaluate a very large number of possible signal combinations.

For example, if:

- 100 intersections exist
- Each intersection has 20 possible timing configurations

The number of possible combinations becomes astronomically large.

Searching every possible combination would require enormous computation time and memory.

Furthermore, traffic conditions continuously change. By the time a traditional algorithm finds the best solution, the traffic pattern may already have changed, making the solution obsolete.

Therefore, traditional search methods are unsuitable for real-time traffic optimization.

(ii) Explain how Hill Climbing would work in this scenario and identify its limitations.

Working of Hill Climbing

Hill Climbing is a local search optimization algorithm.

It begins with an initial traffic signal configuration and continuously makes small adjustments.

Examples include:

- Increasing green time by 5 seconds
- Reducing red signal duration
- Changing traffic signal order
- Coordinating neighboring intersections

After each adjustment, the AI evaluates whether the new configuration improves traffic flow.

If the new solution reduces congestion, it becomes the current solution.

Otherwise, it is rejected.

This process continues until no neighboring solution provides further improvement.

Example

Suppose an intersection initially has:

- Waiting time = 300 seconds

The AI increases the green signal duration.

New waiting time becomes:

- 250 seconds

Since the objective has improved, the new configuration is accepted.

The AI repeats this process until no further reduction in waiting time is possible.

Advantages of Hill Climbing

Fast Optimization

Only neighboring solutions are evaluated.

Therefore, computation is very fast.

Low Memory Requirement

Only the current solution needs to be stored.

Suitable for Continuous Improvement

Signal timings can be updated frequently as traffic changes.

Limitations

May Get Stuck

Hill Climbing only accepts better neighboring solutions.

If none exists, it stops, even if a much better solution exists elsewhere.

No Global Search

The algorithm explores only a small portion of the search space.

Sensitive to Starting Point

Different initial signal timings may produce different final solutions.

Poor Adaptability

In highly dynamic environments, frequent traffic changes may require restarting the search repeatedly.

(iii) Discuss Issues Like Local Maxima, Plateaus, and Ridges with Real-World Interpretation

1. Local Maximum

A local maximum is a solution that is better than all neighboring solutions but is not the best overall solution.

Real-World Example

An intersection experiences moderate traffic flow after adjusting signal timings.

Any small change increases congestion.

Therefore, Hill Climbing stops.

However, another completely different timing plan elsewhere in the city could reduce congestion much more.

The algorithm never discovers it.

2. Plateau

A plateau is a flat search region where many neighboring solutions have identical performance.

Real-World Example

Several traffic signal timing combinations produce the same average waiting time.

Since no neighboring solution appears better, Hill Climbing cannot determine which direction to move.

Progress becomes very slow or completely stops.

3. Ridge

A ridge occurs when the optimal path requires moving through combinations that initially appear worse.

Real-World Example

Reducing congestion at one intersection may temporarily increase waiting time at another intersection.

Eventually, overall traffic flow improves significantly.

However, Hill Climbing rejects the temporary worsening and therefore never reaches the better overall solution.

Summary Table

Problem	Traffic Interpretation	Effect on Hill Climbing
Local Maximum	Good but not best signal timing	Stops too early
Plateau	Many equal signal configurations	No search progress
Ridge	Temporary worsening needed for better solution	Cannot reach global optimum

(iv) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing

Simulated Annealing is an advanced optimization algorithm inspired by the cooling process of metals.

Unlike Hill Climbing, it sometimes accepts worse solutions.

This allows the search to escape local maxima.

Working

Initially,

- High temperature
- Large exploration

Later,

- Temperature decreases
- Search becomes more focused

Advantages

- Escapes local maxima
- Crosses plateaus
- Moves across ridges
- Better global optimization
- Suitable for dynamic traffic environments

Genetic Algorithm (GA)

Genetic Algorithms imitate natural evolution.

Each traffic signal configuration represents a chromosome.

The algorithm maintains a population of possible solutions.

Using evolutionary operators:

- Selection
- Crossover
- Mutation

new generations of better traffic signal timings are produced.

Over many generations, traffic performance improves significantly.

Advantages of Genetic Algorithm

Global Search

Explores many regions simultaneously.

Handles Large Search Space

Suitable for hundreds of intersections.

Parallel Optimization

Multiple candidate solutions evolve together.

Adaptable

New traffic information can easily be incorporated into future generations.

(v) Recommend the Best Approach and Justify Your Choice Based on Scalability and Adaptability

Recommended Algorithm: Genetic Algorithm (GA)

Among all optimization techniques, the **Genetic Algorithm** is the most suitable approach for smart city traffic signal optimization.

Justification

1. Excellent Scalability

A smart city may contain hundreds of intersections.

Genetic Algorithms efficiently search very large solution spaces by evolving multiple candidate solutions simultaneously.

2. Avoids Local Optima

Mutation and crossover allow the algorithm to escape local maxima, plateaus, and ridges.

Therefore, it has a much greater chance of finding the global optimum.

3. Highly Adaptable

Traffic conditions continuously change due to accidents, construction work, weather changes, and special events.

The Genetic Algorithm easily adapts by evolving new signal timing plans using updated traffic information.

4. Multi-Objective Optimization

The algorithm can optimize several objectives simultaneously, including:

- Waiting time
- Fuel consumption
- Traffic congestion
- Carbon emissions
- Emergency vehicle priority

5. Robust Performance

Even if one solution performs poorly, many alternative solutions remain available within the population.

This makes the system reliable in dynamic urban environments.

6. Real-Time Smart City Applications

Modern computing infrastructure, cloud computing, and distributed processing allow Genetic Algorithms to produce high-quality solutions quickly enough for real-time traffic management.

Final Comparison

Feature	Hill Climbing	Simulated Annealing
Search Type	Local Search	Probabilistic Search
Finds Global Optimum	No	Often
Escapes Local Maxima	No	Yes
Handles Large Cities	Moderate	Good
Dynamic Traffic Adaptation	Moderate	Good
Scalability	Moderate	High
Real-Time Performance	Good	Very Good
Feature	Hill Climbing	Simulated Annealing

Conclusion

Smart city traffic signal optimization is a large-scale, dynamic optimization problem involving multiple objectives such as minimizing waiting time, fuel consumption, and traffic congestion. Traditional search algorithms are ineffective because the search space is enormous and traffic conditions change continuously. Hill Climbing provides fast local improvements but suffers from local maxima, plateaus, and ridges, preventing it from finding globally optimal solutions. Simulated Annealing improves performance by occasionally accepting worse solutions, allowing it to escape local optima. However, the Genetic Algorithm is the most suitable approach because it explores multiple solutions simultaneously, scales efficiently to hundreds of intersections, adapts to real-time traffic changes, and effectively optimizes multiple objectives. Therefore, Genetic Algorithms provide the best balance of scalability, adaptability, robustness, and optimization quality for intelligent smart city traffic management systems.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. **Explain why this problem requires an online search agent instead of offline search**
- ii. **Analyze the challenges of partial observability and dynamic environments**
- iii. **Describe how the rover builds and updates its internal model**
- iv. **Compare online search with uninformed and informed search strategies**
- v. **Justify the most suitable approach considering uncertainty, adaptability, and safety**

SOLUTION:

Assignment: Online Search Agent for an Autonomous Mars Rover

Scenario Overview

An autonomous rover is deployed on Mars to explore unknown terrain and collect valuable geological samples. Unlike navigation on Earth, the rover operates in an environment that is unknown, unpredictable, and partially observable. It has limited sensing capability, meaning it can detect only nearby objects such as rocks, craters, or steep slopes, while the rest of the environment remains unknown.

The rover must:

- Navigate safely to collect rock and soil samples.
- Avoid hazards such as craters, large rocks, and steep terrain.
- Operate with limited sensor information.
- Make decisions independently because communication with Earth is delayed.
- Continuously update its knowledge as it explores new regions.

Since the rover does not possess a complete map before starting its mission, it must make decisions in **real time**, making this an ideal application of an **Online Search Agent**.

(i) Explain why this problem requires an Online Search Agent instead of Offline Search.

Online Search Agent

An **Online Search Agent** makes decisions while interacting with the environment. It explores unknown locations, observes new information through its sensors, updates its knowledge, and then decides the next action.

The rover follows the cycle:

1. Observe the surroundings.
2. Update its internal map.
3. Evaluate possible actions.

4. Choose the safest movement.
5. Repeat until the mission is completed.

This allows the rover to react immediately to newly discovered obstacles.

Why Offline Search is Not Suitable

Offline Search assumes that:

- The complete map is already known.
- All obstacles are identified before the search begins.
- The environment remains unchanged during execution.

These assumptions are not valid on Mars because:

- The rover has no complete map before exploration.
- Hidden rocks and craters are discovered only after moving.
- Dust storms and changing terrain may affect navigation.
- Communication delays prevent humans from providing immediate guidance.

Therefore, Offline Search cannot produce an effective route in advance.

Why Online Search is Required

Online Search enables the rover to:

- Learn while exploring.
- React immediately to hazards.
- Change its path when new obstacles appear.
- Continue operating safely even with incomplete information.
- Make autonomous decisions without waiting for instructions from Earth.

Thus, Online Search is the most suitable strategy for autonomous planetary exploration.

(ii) Analyze the Challenges of Partial Observability and Dynamic Environments.

Partial Observability

Partial observability means the rover can observe only a limited area around its current location.

It cannot detect:

- Distant craters
- Hidden rocks
- Deep valleys beyond sensor range
- Dangerous slopes behind hills

As a result, the rover must make decisions with incomplete information.

Example

The rover plans to move forward because the path appears safe.

After moving closer, its sensors detect a large crater that was previously invisible.

The rover must immediately change its route.

Dynamic Environment

Although Mars changes more slowly than Earth, the environment is still dynamic.

Possible changes include:

- Dust storms reducing visibility.
- Loose sand causing wheel slippage.
- Falling rocks.
- Changing lighting conditions affecting sensors.
- Newly detected hazards as exploration continues.

The rover must continuously adapt its navigation strategy.

Challenges Faced

Limited Sensor Range

Only nearby terrain can be observed.

Uncertain Information

Sensor readings may contain noise or errors.

Communication Delay

Commands from Earth may take several minutes to arrive.

The rover must therefore make independent decisions.

Energy Constraints

Frequent route changes increase battery usage.

Efficient navigation is essential.

Safety Requirements

Collisions or falls into craters could permanently damage the rover and end the mission.

(iii) Describe how the rover builds and updates its internal model.

Internal Model

The rover maintains an **internal map** representing everything it has discovered. Initially, this map is almost empty because the environment is unknown.

Step 1: Initial Exploration

The rover starts with only:

- Current position
- Mission objective
- Sensor information

Unknown regions remain unexplored.

Step 2: Sense the Environment

Using cameras, LiDAR, radar, and proximity sensors, the rover identifies:

- Rocks
- Craters
- Sand dunes
- Safe pathways
- Sample locations

Step 3: Update the Internal Map

Newly discovered information is added to the rover's memory.

Examples include:

- Safe paths
- Dangerous obstacles
- Visited locations
- Unexplored regions
- Sample collection sites

Step 4: Plan the Next Move

The rover evaluates all possible neighboring positions.

It selects the safest and most efficient path based on its updated knowledge.

Step 5: Continuous Learning

As exploration continues, the internal map becomes increasingly accurate.

Future decisions become more informed, reducing unnecessary exploration and improving mission efficiency.

(iv) Compare Online Search with Uninformed and Informed Search Strategies.

Uninformed Search

Examples include:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

Characteristics

- No heuristic information.
- Assumes complete knowledge of the search space.
- Explores systematically.

Advantages

- Simple implementation.
- Guarantees completeness in many cases.

Disadvantages

- High computational cost.
- Inefficient for unknown environments.
- Cannot react effectively to newly discovered obstacles.

Informed Search

Examples include:

- Greedy Best-First Search
- A* Search

Characteristics

- Uses heuristic estimates.
- Searches more efficiently than uninformed methods.

Advantages

- Faster route planning.
- Lower search cost.
- Better path quality.

Disadvantages

- Requires reliable heuristic information.
- Usually assumes a mostly known environment.

- Performance decreases when the map is incomplete.

Online Search

Characteristics

- Learns while moving.
- Continuously updates knowledge.
- Makes decisions using current sensor data.

Advantages

- Suitable for unknown environments.
- Handles partial observability.
- Adapts to newly discovered obstacles.
- Supports autonomous operation.

Disadvantages

- May temporarily choose non-optimal routes.
- Requires continuous computation.
- Performance depends on sensor accuracy.

Comparison Table

Feature	Uninformed Search	Informed Search
Complete Map Required	Yes	Usually Yes
Uses Heuristic	No	Yes
Learns While Exploring	No	No
Handles Unknown Terrain	Poor	Moderate
Adapts to New Obstacles	No	Limited
Real-Time Decision Making	Poor	Moderate
Suitable for Mars Rover	No	Partially

(v) Justify the Most Suitable Approach Considering Uncertainty, Adaptability, and Safety.

Recommended Approach: Online Search Agent

The **Online Search Agent** is the most suitable approach for autonomous Mars exploration.

Justification

1. Handles Uncertainty

The rover never has complete information.

Online Search allows decisions to be made using the latest sensor observations instead of relying on an inaccurate precomputed map.

2. High Adaptability

As new obstacles or safe routes are discovered, the rover immediately updates its internal model and changes its path.

This ensures successful navigation in unpredictable environments.

3. Improved Safety

The rover continuously monitors nearby terrain.

Dangerous regions such as craters or unstable slopes are avoided as soon as they are detected.

This greatly reduces the risk of mission failure.

4. Autonomous Decision Making

Communication delays between Earth and Mars make remote control impractical.

Online Search enables the rover to operate independently and respond instantly to changing conditions.

5. Efficient Exploration

The rover gradually builds an accurate internal map.

Previously explored safe paths are reused, reducing unnecessary travel and conserving battery power.

6. Real-Time Performance

Online Search supports continuous sensing, planning, and action.

This makes it ideal for autonomous exploration where conditions change during the mission.

Final Comparison

Criteria	Offline Search	Uninformed Search	Informed Search	Online Search
-----------------	-----------------------	--------------------------	------------------------	----------------------

Requires Complete Map	Yes	Yes	Usually Yes	No
Handles Partial Observability	No	No	Limited	Excellent
Adapts to Dynamic Environment	No	Poor	Moderate	Excellent
Real-Time Decision Making	No	Poor	Moderate	Excellent
Autonomous Operation	Limited	Limited	Good	Excellent
Safety	Moderate	Moderate	Good	Excellent
Recommended for Mars Rover	No	No	Partially	Best Choice

CONCLUSION:

Exploring Mars is a real-time decision-making problem because the rover operates in an unknown and partially observable environment where hazards are discovered only during exploration. Offline, uninformed, and informed search algorithms assume that the environment is already known and therefore cannot effectively manage unexpected obstacles or changing conditions. In contrast, an Online Search Agent continuously senses its surroundings, updates an internal map, and makes intelligent navigation decisions based on the latest information. This enables the rover to safely avoid hazards, adapt to uncertainty, conserve energy, and complete its scientific mission without human intervention. Therefore, the Online Search Agent is the most suitable approach for autonomous Mars exploration, providing the best balance of uncertainty handling, adaptability, efficiency, and operational safety.

4.Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled

before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation**
- ii. Explain how backtracking search works in this scenario**
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency**
- iv. Analyze real-world challenges and justify the best strategy for scalability**

SOLUTION:

Assignment: Constraint Satisfaction Problem (CSP) for Automated University Exam Timetable Generation

Scenario Overview

A university plans to develop an Artificial Intelligence (AI)-based Exam Timetable Generation System to automatically schedule examinations for all courses. The system must satisfy several constraints while ensuring fairness and efficient utilization of university resources.

The system must ensure that:

- No student has overlapping examinations.
- The number of available examination halls is limited.
- Some subjects must be conducted before others due to academic requirements.
- Faculty members are available only during specific time slots.
- Students requiring special accommodations receive suitable examination arrangements.
- The scheduling process remains efficient even as the number of students and courses increases.

Since many constraints must be satisfied simultaneously, this problem is best modeled as a **Constraint Satisfaction Problem (CSP)**.

(i) Formulate the Problem as a Constraint Satisfaction Problem (CSP)

What is a Constraint Satisfaction Problem?

A **Constraint Satisfaction Problem (CSP)** is an Artificial Intelligence problem in which values must be assigned to a set of variables while satisfying all specified constraints.

A CSP consists of three components:

- Variables
- Domains
- Constraints

The objective is to find a valid assignment that satisfies every constraint.

For exam timetable generation:

- Variables represent exams.
- Domains represent available time slots and examination halls.
- Constraints ensure that conflicts do not occur.

(ii) Clearly Define Variables, Domains, and Constraints with Explanation

1. Variables

Variables represent the examinations that must be scheduled.

Suppose the university offers the following subjects:

- Mathematics (M)
- Physics (P)
- Chemistry (C)
- Computer Science (CS)
- English (E)

Each subject is represented as a variable.

Example:

- X_1 = Mathematics Exam
- X_2 = Physics Exam
- X_3 = Chemistry Exam
- X_4 = Computer Science Exam
- X_5 = English Exam

Each variable requires an assignment consisting of a **time slot** and an **exam hall**.

2. Domains

The domain represents all possible values that can be assigned to each variable.

Time Slots

Example:

- T_1 = Monday Morning
- T_2 = Monday Afternoon
- T_3 = Tuesday Morning

- T4 = Tuesday Afternoon
- T5 = Wednesday Morning

Examination Halls

Example:

- Hall A
- Hall B
- Hall C

Therefore, each variable has a domain such as:

(Mathematics)

$\{(T1, \text{Hall A}), (T1, \text{Hall B}), (T2, \text{Hall A}), (T2, \text{Hall B}), \dots\}$

The AI system selects one valid combination from this domain.

3. Constraints

The solution must satisfy all constraints.

Constraint 1: No Student Should Have Overlapping Exams

If a student is enrolled in both Mathematics and Physics, those exams cannot be scheduled at the same time.

Example:

Mathematics $\neq$ Physics (same time slot)

This is the most important constraint.

Constraint 2: Limited Number of Examination Halls

Only a limited number of halls are available.

If there are only three halls, no more than three examinations can occur simultaneously.

Constraint 3: Subject Ordering Constraints

Some subjects must be conducted before others.

Example:

Practical Examination $\rightarrow$ Theory Examination

or

Computer Networks must occur before Network Lab.

Constraint 4: Faculty Availability

Faculty members may only be available during certain periods.

Example:

Professor A

Available:

- Monday Morning
- Tuesday Afternoon

Unavailable:

- Wednesday Morning

The examination must therefore be scheduled only during available periods.

Constraint 5: Special Accommodation

Some students require:

- Separate examination rooms
- Extra writing time
- Wheelchair-accessible halls
- Assistive technologies

The assigned hall must satisfy these requirements.

Constraint 6: Hall Capacity

Each hall has limited seating.

Example:

Hall A

Capacity = 120 students

If 180 students register for a course, Hall A alone cannot accommodate them.

Constraint 7: University Policies

Examples include:

- Maximum two exams per student per day.
- Lunch break between sessions.
- No examinations on university holidays.
- Minimum preparation gap between major examinations.

Summary of CSP Components

Component Description

Variables	Exams (Mathematics, Physics, Chemistry, etc.)
Domains	Available time slots and examination halls
Constraints	Student conflicts, hall limits, faculty availability, subject order, special accommodations, hall capacity, university policies

(iii) Explain How Backtracking Search Works in This Scenario

What is Backtracking Search?

Backtracking Search is a systematic CSP-solving algorithm that assigns values to variables one at a time.

If an assignment violates any constraint, the algorithm goes back (backtracks) and tries another assignment.

Working Steps

Step 1

Choose an unscheduled examination.

Example:

Mathematics

Assign

Monday Morning

Hall A

Check constraints.

No conflicts found.

Accept.

Step 2

Schedule Physics.

Attempt:

Monday Morning

Hall A

Conflict detected:

A student has both Mathematics and Physics.

Assignment rejected.

Backtrack.

Try:

Tuesday Morning

Hall B

No conflicts.

Accept.

Step 3

Continue scheduling remaining examinations.

Each assignment is checked immediately against all constraints.

Step 4

If a later assignment becomes impossible,
the algorithm returns to the previous decision,
changes it,
and continues searching.

This process continues until every examination has a valid assignment.

Advantages

- Guarantees a complete solution if one exists.
- Avoids unnecessary invalid schedules.
- Simple and widely used for CSPs.

Limitation

Without optimization, Backtracking may explore many unnecessary possibilities as the number of exams increases.

(iv) Discuss How Constraint Propagation (Forward Checking) Improves Efficiency

Constraint Propagation

Constraint propagation reduces the search space before conflicts occur.

One common technique is **Forward Checking**.

Forward Checking

Whenever a variable receives a value,

the algorithm immediately removes inconsistent values from the domains of neighboring variables.

Example

Suppose:

Mathematics

Assigned

Monday Morning

Immediately remove Monday Morning from the domains of:

- Physics
- Chemistry
- Computer Science

if students are enrolled in both subjects.

Only valid time slots remain.

Benefits

Early Conflict Detection

Impossible schedules are identified immediately.

Smaller Search Space

Many invalid assignments are removed before exploration.

Faster Scheduling

The algorithm performs significantly fewer backtracking operations.

Better Scalability

Forward Checking remains efficient even when the number of courses increases.

Comparison

Without Forward Checking

Many invalid assignments explored

Frequent backtracking

Higher execution time

With Forward Checking

Invalid assignments removed early

Much less backtracking

Faster execution

Without Forward Checking

Larger search space

With Forward Checking

Reduced search space

(v) Analyze Real-World Challenges and Justify the Best Strategy for Scalability**Real-World Challenges****Large Number of Courses**

Universities may conduct hundreds of examinations.

The search space becomes extremely large.

Thousands of Students

Student enrollment creates many overlapping subject combinations.

Conflict checking becomes more complex.

Dynamic Changes

Unexpected events may occur, such as:

- Faculty leave
- Hall maintenance
- Public holidays
- Emergency timetable changes

The system must quickly generate a revised timetable.

Resource Constraints

Limited examination halls, invigilators, and equipment increase scheduling complexity.

Special Accommodation Requirements

Students with disabilities require personalized scheduling and accessible facilities.

Best Strategy for Scalability

The most effective approach is:

Backtracking Search combined with Constraint Propagation (Forward Checking).

Additional heuristics further improve performance, such as:

- **Minimum Remaining Values (MRV):** Select the exam with the fewest remaining valid slots first.

- **Degree Heuristic:** Schedule the exam that conflicts with the largest number of other exams first.
- **Least Constraining Value (LCV):** Choose the assignment that leaves the greatest flexibility for remaining exams.

Justification

1. Efficient Search

Backtracking guarantees a correct timetable whenever a solution exists.

2. Reduced Computation

Forward Checking removes invalid assignments before they cause conflicts.

3. Better Scalability

Heuristics significantly reduce the number of explored states, allowing the algorithm to handle hundreds of courses and thousands of students.

4. High Adaptability

If faculty availability or hall assignments change, the CSP can be updated and solved again without rebuilding the entire timetable manually.

5. Practical Implementation

Most modern university timetable systems are based on CSP techniques because they provide reliable, scalable, and maintainable solutions.

Final Comparison

Feature	Simple Backtracking	Backtracking + Forward Checking + Heuristics
Correct Solution	Yes	Yes
Search Speed	Moderate	High
Backtracking Operations	Many	Few
Scalability	Moderate	Excellent
Handles Large Universities	Difficult	Efficient
Practical Usage	Limited	Widely Used

Overall Recommendation	Good	Best Choice
Feature	Simple Backtracking	Backtracking + Forward Checking + Heuristics

Conclusion

The university exam timetable generation problem is a classic Constraint Satisfaction Problem (CSP) because examinations must be assigned to time slots and halls while satisfying numerous academic and administrative constraints. The CSP is defined using variables (exams), domains (available time slots and halls), and constraints (student conflicts, hall capacity, faculty availability, subject precedence, and special accommodations). Backtracking Search systematically explores possible assignments and guarantees a valid timetable when one exists, while Constraint Propagation using Forward Checking greatly improves efficiency by eliminating invalid assignments before deeper exploration. When combined with heuristics such as Minimum Remaining Values (MRV), Degree Heuristic, and Least Constraining Value (LCV), the system becomes highly scalable and capable of efficiently generating conflict-free timetables for universities with thousands of students and hundreds of courses. Therefore, Backtracking Search enhanced with Forward Checking and CSP heuristics is the most suitable strategy for automated university exam timetable generation, providing the best balance of correctness, efficiency, scalability, and adaptability.

5.Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem

- ii. **Analyze the computational challenges of large game trees**
- iii. **Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**
- iv. **Design an evaluation function and justify the factors considered**
- v. **Recommend a strategy for balancing optimality and real-time performance**

SOLUTION:

Assignment: Strategic Game AI for Real-Time Decision Making Using Minimax and Alpha–Beta Pruning

Scenario Overview

A gaming company is developing an **Artificial Intelligence (AI) opponent** for a real-time strategy (RTS) game. The AI competes against human players and must make intelligent decisions within a very short time. At every stage of the game, the AI has many possible actions such as moving units, attacking enemies, defending bases, collecting resources, or constructing buildings.

The AI must:

- Compete effectively against human players.
- Make decisions within strict real-time limits.
- Evaluate complex game states with many possible moves.
- Handle an extremely large decision tree while maintaining high performance.

Since every AI move is followed by an opponent's move, the problem is naturally modeled as an adversarial search problem, where one player attempts to maximize the outcome while the opponent attempts to minimize it. The Minimax algorithm and Alpha–Beta Pruning are widely used to solve such problems efficiently.

(i) Explain How the Minimax Algorithm Models This Problem

What is the Minimax Algorithm?

The Minimax algorithm is an adversarial search algorithm used in two-player zero-sum games. It assumes that both players make optimal decisions.

- **MAX Player:** The AI tries to maximize its chance of winning.
- **MIN Player:** The human opponent tries to minimize the AI's advantage.

The algorithm builds a game tree in which:

- Each node represents a game state.
- Each branch represents a possible move.

- Leaf nodes are evaluated using an evaluation function.

The AI selects the move that provides the highest guaranteed outcome, assuming the opponent always plays optimally.

Working of Minimax in the Strategy Game

Step 1: Generate Possible Moves

The AI generates all legal actions from the current game state.

Examples:

- Attack enemy base.
- Defend friendly units.
- Gather resources.
- Build new structures.
- Train additional soldiers.

Step 2: Simulate Opponent Moves

For every AI move, the algorithm predicts the opponent's best possible response.

Step 3: Continue Expanding the Game Tree

The algorithm alternates between:

- MAX level (AI's turn)
- MIN level (Opponent's turn)

until a predefined search depth or terminal game state is reached.

Step 4: Evaluate Leaf Nodes

Each final game state is assigned a numerical score.

Example:

- Win = +1000
- Draw = 0
- Loss = -1000

Intermediate game states are evaluated using an evaluation function.

Step 5: Propagate Values Upward

- MAX nodes choose the **highest** value.
- MIN nodes choose the **lowest** value.

The value reaching the root determines the AI's best move.

Advantages of Minimax

- Produces rational decisions.
- Considers the opponent's best possible response.
- Guarantees the optimal move if the entire game tree is explored.

Limitations

- Very expensive computationally.
- Not practical for games with extremely large search spaces.
- Decision time increases rapidly as the search depth grows.

(ii) Analyze the Computational Challenges of Large Game Trees

Real-time strategy games have an enormous number of possible actions.

Suppose:

- Each position has 30 possible moves.
- The AI searches 6 levels deep.

The total number of nodes becomes:

$$30^6 = 729,000,000 \text{ nodes}$$

Searching every node is computationally infeasible in real time.

Major Challenges

1. Exponential Growth

Each additional search level multiplies the number of states dramatically.

This phenomenon is known as the **combinatorial explosion**.

2. Time Constraints

The AI may have only a few milliseconds to make each decision.

Searching the entire tree is impossible within this time.

3. Memory Consumption

Every game state requires memory.

Large trees quickly consume available RAM.

4. Dynamic Game Environment

Players continuously change strategies.

The game state changes before a complete search finishes.

5. Multiple Decision Variables

The AI simultaneously decides:

- Unit movement
- Resource allocation
- Combat strategy
- Base construction
- Defensive positioning

Each additional decision increases the branching factor.

(iii) Explain How Alpha–Beta Pruning Improves Efficiency with Detailed Reasoning

What is Alpha–Beta Pruning?

Alpha–Beta Pruning is an optimization technique for the Minimax algorithm.

It reduces the number of nodes explored without changing the final decision.

The algorithm eliminates branches that cannot influence the optimal result.

Alpha Value (α)

Alpha represents the **best score that the MAX player can guarantee** so far.

Initially:

$$\alpha = -\infty$$

Whenever MAX finds a better value, alpha is updated.

Beta Value (β)

Beta represents the **best score that the MIN player can guarantee** so far.

Initially:

$$\beta = +\infty$$

Whenever MIN finds a lower value, beta is updated.

Pruning Condition

If at any point:

$$\alpha \geq \beta$$

the remaining branches under that node cannot improve the final decision.

Therefore, those branches are discarded without evaluation.

This is called **Alpha–Beta Pruning**.

Example

Suppose the AI has already found a move with a score of **80**.

While exploring another branch, the opponent discovers a response limiting the AI to **50**.

Since the AI already has a guaranteed score of **80**, exploring the branch that can only produce **50** cannot improve the result.

The branch is immediately pruned.

Advantages of Alpha–Beta Pruning

Faster Search

Many unnecessary nodes are ignored.

Deeper Search

Time saved through pruning allows the AI to examine more future moves.

Same Optimal Decision

Although fewer nodes are explored, the final move selected is exactly the same as standard Minimax.

Better Real-Time Performance

The AI responds much faster, making Alpha–Beta Pruning ideal for strategy games.

(iv) Design an Evaluation Function and Justify the Factors Considered

Since exploring the entire game tree is impossible, the AI evaluates non-terminal game states using an **evaluation function**.

The function estimates how favorable a game state is for the AI.

A possible evaluation function is:

Evaluation Score =

$(10 \times \text{Resources}) +$

$(20 \times \text{Army Strength}) +$

$(15 \times \text{Base Health}) +$

$(10 \times \text{Map Control}) +$
 $(25 \times \text{Strategic Position}) -$
 $(20 \times \text{Enemy Strength})$

Factors Considered

1. Resources

More resources allow the AI to build stronger units and structures.

Higher resources increase the evaluation score.

2. Army Strength

The number and quality of military units directly influence combat success.

A stronger army results in a higher score.

3. Base Health

Protecting the main base is essential for survival.

Higher base health improves the evaluation value.

4. Map Control

Controlling important regions provides tactical and economic advantages.

The AI rewards positions with greater map dominance.

5. Strategic Position

Units occupying high ground, defensive positions, or key objectives receive higher scores.

6. Enemy Strength

A powerful opponent increases the level of threat.

Therefore, greater enemy strength decreases the evaluation score.

Justification

These factors represent the most important elements of success in a real-time strategy game.

Combining them enables the AI to evaluate positions realistically, even when the game has not yet ended.

(v) Recommend a Strategy for Balancing Optimality and Real-Time Performance

Recommended Strategy

The best practical approach is to combine:

- Minimax Algorithm
- Alpha–Beta Pruning
- Depth-Limited Search
- Heuristic Evaluation Function
- Move Ordering

Justification

1. Optimal Decision Quality

Minimax ensures the AI considers both its own moves and the opponent's best responses.

2. Faster Computation

Alpha–Beta Pruning removes unnecessary branches, significantly reducing computation time.

3. Real-Time Performance

Depth-limited search ensures decisions are completed within strict time limits.

4. Efficient Position Evaluation

A heuristic evaluation function estimates the quality of non-terminal game states without exploring the entire tree.

5. Better Pruning

Ordering promising moves first increases the effectiveness of Alpha–Beta Pruning, allowing even more branches to be eliminated.

6. Scalability

This combination scales well to increasingly complex games while maintaining acceptable response times.

Final Comparison

Feature	Minimax	Alpha–Beta Pruning
---------	---------	--------------------

Search Type	Adversarial Search	Optimized Adversarial Search
Finds Optimal Move	Yes	Yes
Number of Nodes Explored	Very Large	Much Smaller
Time Complexity	High	Significantly Reduced
Memory Usage	High	Lower
Suitable for Real-Time Games	Limited	Excellent
Practical Industry Usage	Rare Alone	Widely Used

Conclusion

Developing an AI opponent for a real-time strategy game is a challenging adversarial search problem due to the enormous number of possible game states and strict time constraints. The Minimax algorithm provides a strong decision-making framework by assuming optimal play from both the AI and the human opponent, but its computational cost grows exponentially with search depth. Alpha–Beta Pruning addresses this limitation by eliminating branches that cannot affect the final decision, allowing the AI to search deeper while producing the same optimal result. A carefully designed evaluation function, considering factors such as resources, army strength, base health, map control, strategic position, and enemy strength, enables effective assessment of non-terminal states. For real-time performance, the most suitable strategy is to combine Minimax, Alpha–Beta Pruning, depth-limited search, heuristic evaluation, and efficient move ordering, providing an excellent balance between optimal decision quality, computational efficiency, scalability, and responsiveness.